

TutorHub

Online Tutoring Platform

FULL 30-DAY PROFESSIONAL DEVELOPMENT SCHEDULE

Backend (ASP.NET Core) + Frontend (React) | 2 Hours Daily | April 20 — May 20, 2026

HASEEB

Backend Developer
ASP.NET Core Web API

TALHA

Frontend Developer
React + Vite

Project Start
Apr 20, 2026

Daily Hours
2 hrs/person

Target Launch
May 20, 2026

Deployment
Azure Cloud

How to Use This Document

This document is your complete step-by-step guide for building TutorHub from scratch in 30 days. It is divided into two major parts:

- PART A — Backend (Haseeb): All ASP.NET Core steps, folder structures, class names, endpoint names, JWT security, database models, and package installations.
- PART B — Frontend (Talha): All React steps, folder structures, component names, page routes, API integration, node module installations, and state management.

Each week contains a daily task breakdown. Both team members work independently for 2 hours each day on their respective technology. At the end of each week, integration points are defined where Haseeb and Talha sync and test together.

Legend

Backend (Haseeb) — Blue	Frontend (Talha) — Green
ASP.NET Core 8 Web API	React 18 + Vite
Entity Framework Core	Axios for API calls
SQL Server	Zustand for state
JWT Authentication	React Router v6
SignalR (Chat)	Tailwind CSS
Azure Blob Storage	React Query
Stripe.NET	Stripe.js
Swagger / OpenAPI	React Hook Form

PART A — BACKEND

Haseeb | ASP.NET Core 8 Web API | 2 Hours Daily

Section 1 — Backend Project Setup & Architecture

1.1 Tools and Software to Install

Before writing a single line of code, Haseeb must install the following tools on his machine:

Tool	Version / Edition	Purpose
.NET SDK	8.0 LTS	ASP.NET Core runtime and CLI
Visual Studio 2022	Community Edition	Main IDE for backend
SQL Server	2022 Developer Edition	Local database engine
SQL Server Management Studio (SSMS)	Latest	Database GUI client
Postman	Latest	API testing
Git	Latest	Version control
Azure CLI	Latest	Cloud deployment
Node.js	18+ LTS	Required for Swagger UI tooling
Docker Desktop	Latest	Container deployment (optional but recommended)

1.2 Create the Solution and Project

Open a terminal (Command Prompt or PowerShell) and run the following commands exactly in order. Each command is explained below it.

Step 1 — Create a solution folder:

- Command: `mkdir TutorHub`
- Command: `cd TutorHub`
- Command: `dotnet new sln -n TutorHub`
- This creates the Solution file (TutorHub.sln) that holds all projects together.

Step 2 — Create the Web API project:

- Command: `dotnet new webapi -n TutorHub.API --framework net8.0`
- Command: `dotnet sln add TutorHub.API/TutorHub.API.csproj`

Step 3 — Create class library projects (Clean Architecture layers):

- Command: `dotnet new classlib -n TutorHub.Core`
- Command: `dotnet new classlib -n TutorHub.Infrastructure`
- Command: `dotnet new classlib -n TutorHub.Application`
- Command: `dotnet sln add TutorHub.Core/TutorHub.Core.csproj`
- Command: `dotnet sln add TutorHub.Infrastructure/TutorHub.Infrastructure.csproj`
- Command: `dotnet sln add TutorHub.Application/TutorHub.Application.csproj`

Step 4 — Add project references:

- Command: `cd TutorHub.API && dotnet add reference ../TutorHub.Application/TutorHub.Application.csproj`
- Command: `cd ../TutorHub.Application && dotnet add reference ../TutorHub.Core/TutorHub.Core.csproj`
- Command: `cd ../TutorHub.Infrastructure && dotnet add reference ../TutorHub.Core/TutorHub.Core.csproj`
- Command: `cd ../TutorHub.API && dotnet add reference ../TutorHub.Infrastructure/TutorHub.Infrastructure.csproj`

1.3 Complete Folder and File Directory Structure

The entire backend must follow this exact Clean Architecture folder structure. Each folder and file is listed below with its purpose.

TutorHub.Core (Domain Layer — No Dependencies)

This layer contains only business domain objects. It must NEVER reference any other project.

Folder Path	File Name	Purpose
TutorHub.Core/Entities/	ApplicationUser.cs	Base user entity (extends IdentityUser)
TutorHub.Core/Entities/	TutorProfile.cs	Tutor profile details
TutorHub.Core/Entities/	StudentProfile.cs	Student profile details
TutorHub.Core/Entities/	ParentProfile.cs	Parent profile and child links
TutorHub.Core/Entities/	Subject.cs	Subjects taught on platform
TutorHub.Core/Entities/	GradeLevel.cs	Grade levels available
TutorHub.Core/Entities/	TutorSubject.cs	Many-to-many: tutor to subjects
TutorHub.Core/Entities/	TutorAvailability.cs	Tutor weekly availability slots
TutorHub.Core/Entities/	TutorDocument.cs	Uploaded verification documents
TutorHub.Core/Entities/	Booking.cs	Session booking entity
TutorHub.Core/Entities/	Payment.cs	Payment transaction record
TutorHub.Core/Entities/	Review.cs	Student review for tutor
TutorHub.Core/Entities/	ChatMessage.cs	Chat message entity
TutorHub.Core/Entities/	ChatConversation.cs	Chat conversation between users
TutorHub.Core/Entities/	Notification.cs	In-app notification entity
TutorHub.Core/Entities/	Payout.cs	Tutor payout records
TutorHub.Core/Entities/	RefreshToken.cs	JWT refresh token entity
TutorHub.Core/Entities/	AuditLog.cs	Admin action audit logs
TutorHub.Core/Enums/	UserRole.cs	Enum: Student, Parent, Tutor, Admin
TutorHub.Core/Enums/	BookingStatus.cs	Enum: Pending, Confirmed, Cancelled, Completed
TutorHub.Core/Enums/	TeachingMode.cs	Enum: Online, InPerson, Both
TutorHub.Core/Enums/	DocumentStatus.cs	Enum: Pending, Approved, Rejected

TutorHub.Core/Enums/	PayoutStatus.cs	Enum: Pending, Processing, Completed, Failed
TutorHub.Core/Interfaces/	IUserRepository.cs	User data access interface
TutorHub.Core/Interfaces/	ITutorRepository.cs	Tutor data access interface
TutorHub.Core/Interfaces/	IBookingRepository.cs	Booking data access interface
TutorHub.Core/Interfaces/	IPaymentRepository.cs	Payment data access interface
TutorHub.Core/Interfaces/	IReviewRepository.cs	Review data access interface
TutorHub.Core/Interfaces/	IChatRepository.cs	Chat data access interface
TutorHub.Core/Interfaces/	INotificationRepository.cs	Notification data access interface
TutorHub.Core/Interfaces/	IEmailService.cs	Email service interface
TutorHub.Core/Interfaces/	IFileStorageService.cs	File upload service interface
TutorHub.Core/Interfaces/	IPaymentService.cs	Payment processing interface
TutorHub.Core/Interfaces/	ITokenService.cs	JWT token generation interface

TutorHub.Core — Entity Property Details

Every database entity class and its properties are listed below. These are the exact properties you will code inside each .cs file.

Entity Class	Property Name : Type	Notes
ApplicationUser.cs	Id : string (GUID)	Inherits from IdentityUser
ApplicationUser.cs	FirstName : string	Required
ApplicationUser.cs	LastName : string	Required
ApplicationUser.cs	Role : UserRole (enum)	Student / Parent / Tutor / Admin
ApplicationUser.cs	IsActive : bool	Default true
ApplicationUser.cs	IsEmailVerified : bool	Default false
ApplicationUser.cs	EmailVerificationToken : string?	Nullable
ApplicationUser.cs	CreatedAt : DateTime	Auto set on create
ApplicationUser.cs	LastLoginAt : DateTime?	Updated on login
TutorProfile.cs	Id : int	Primary key
TutorProfile.cs	UserId : string	FK to ApplicationUser
TutorProfile.cs	Headline : string	Short bio title
TutorProfile.cs	Bio : string	Full description
TutorProfile.cs	Education : string	Qualification string
TutorProfile.cs	ExperienceYears : int	Number of years
TutorProfile.cs	City : string	Location city
TutorProfile.cs	HourlyRateMin : decimal	Minimum rate
TutorProfile.cs	HourlyRateMax : decimal	Maximum rate

TutorProfile.cs	TeachingMode : TeachingMode (enum)	Online/InPerson/Both
TutorProfile.cs	ProfileImageUrl : string?	Azure Blob URL
TutorProfile.cs	IsVerified : bool	Set by admin after doc approval
TutorProfile.cs	ApprovalStatus : DocumentStatus (enum)	Pending/Approved/Rejected
TutorProfile.cs	StripeConnectAccountId : string?	For payouts
TutorProfile.cs	AverageRating : decimal	Computed from reviews
TutorProfile.cs	TotalReviews : int	Count of reviews
Booking.cs	Id : int	Primary key
Booking.cs	StudentUserId : string	FK to ApplicationUser
Booking.cs	TutorUserId : string	FK to ApplicationUser
Booking.cs	SubjectId : int	FK to Subject
Booking.cs	ScheduledDate : DateTime	Date of session
Booking.cs	StartTime : TimeSpan	Session start time
Booking.cs	DurationMinutes : int	30, 60, 90, or 120
Booking.cs	TeachingMode : TeachingMode	Online or InPerson
Booking.cs	Status : BookingStatus	Pending/Confirmed/Cancelled/Completed
Booking.cs	TotalAmount : decimal	Tutor fee + platform fee
Booking.cs	PlatformFee : decimal	10% of tutor fee
Booking.cs	TutorEarning : decimal	Amount tutor receives
Booking.cs	StudentNotes : string?	Optional notes from student
Booking.cs	CancellationReason : string?	If cancelled
Booking.cs	MeetingLink : string?	Video call URL for online sessions
Booking.cs	CreatedAt : DateTime	Booking request time
Payment.cs	Id : int	Primary key
Payment.cs	BookingId : int	FK to Booking
Payment.cs	StudentUserId : string	FK to ApplicationUser
Payment.cs	StripePaymentIntentId : string	Stripe reference
Payment.cs	Amount : decimal	Total charged
Payment.cs	Currency : string	Default USD
Payment.cs	Status : string	Stripe payment status
Payment.cs	RefundAmount : decimal?	If refunded
Payment.cs	CreatedAt : DateTime	Payment time
Review.cs	Id : int	Primary key

Review.cs	BookingId : int	FK to Booking
Review.cs	StudentUserId : string	Reviewer
Review.cs	TutorUserId : string	Reviewed tutor
Review.cs	Rating : int	1 to 5
Review.cs	Comment : string?	Text review
Review.cs	CreatedAt : DateTime	Submitted date
ChatMessage.cs	Id : int	Primary key
ChatMessage.cs	ConversationId : int	FK to ChatConversation
ChatMessage.cs	SenderId : string	FK to ApplicationUser
ChatMessage.cs	Content : string	Message text
ChatMessage.cs	FileUrl : string?	Attached file if any
ChatMessage.cs	IsRead : bool	Read receipt
ChatMessage.cs	SentAt : DateTime	Message timestamp
TutorAvailability.cs	Id : int	Primary key
TutorAvailability.cs	TutorUserId : string	FK to ApplicationUser
TutorAvailability.cs	DayOfWeek : DayOfWeek (enum)	Monday to Sunday
TutorAvailability.cs	StartTime : TimeSpan	Slot start
TutorAvailability.cs	EndTime : TimeSpan	Slot end
TutorAvailability.cs	IsBlocked : bool	If tutor blocked this slot

TutorHub.Application (Business Logic Layer)

Folder Path	File Name	Purpose
TutorHub.Application/DTOs/Auth/	RegisterRequestDto.cs	Registration payload from client
TutorHub.Application/DTOs/Auth/	LoginRequestDto.cs	Login payload
TutorHub.Application/DTOs/Auth/	AuthResponseDto.cs	JWT token response
TutorHub.Application/DTOs/Auth/	ForgotPasswordDto.cs	Email for password reset
TutorHub.Application/DTOs/Auth/	ResetPasswordDto.cs	New password + token
TutorHub.Application/DTOs/Tutor/	TutorProfileDto.cs	Full tutor profile for clients
TutorHub.Application/DTOs/Tutor/	TutorProfileUpdateDto.cs	Update payload
TutorHub.Application/DTOs/Tutor/	TutorSearchResultDto.cs	Card view for search results
TutorHub.Application/DTOs/Tutor/	TutorAvailabilityDto.cs	Availability slots payload
TutorHub.Application/DTOs/Tutor/	TutorEarningsDto.cs	Earnings summary payload
TutorHub.Application/DTOs/Booking/	CreateBookingDto.cs	New booking payload
TutorHub.Application/DTOs/Booking/	BookingResponseDto.cs	Full booking details
TutorHub.Application/DTOs/Booking/	CancelBookingDto.cs	Cancellation reason
TutorHub.Application/DTOs/Payment/	CreatePaymentIntentDto.cs	Stripe payment intent request

TutorHub.Application/DTOs/Payment/	PaymentResponseDto.cs	Payment confirmation
TutorHub.Application/DTOs/Review/	CreateReviewDto.cs	Review submission payload
TutorHub.Application/DTOs/Review/	ReviewResponseDto.cs	Review data for display
TutorHub.Application/DTOs/Chat/	SendMessageDto.cs	New chat message payload
TutorHub.Application/DTOs/Chat/	ChatMessageDto.cs	Message display payload
TutorHub.Application/DTOs/Admin/	UserListDto.cs	User management list view
TutorHub.Application/DTOs/Admin/	TutorApprovalDto.cs	Approval decision payload
TutorHub.Application/DTOs/Admin/	PlatformStatsDto.cs	Dashboard analytics
TutorHub.Application/Services/	AuthService.cs	Registration, login, JWT logic
TutorHub.Application/Services/	TutorService.cs	Profile, docs, availability logic
TutorHub.Application/Services/	BookingService.cs	Booking workflow logic
TutorHub.Application/Services/	PaymentService.cs	Stripe integration logic
TutorHub.Application/Services/	ReviewService.cs	Review submission logic
TutorHub.Application/Services/	ChatService.cs	Chat history and messages
TutorHub.Application/Services/	SearchService.cs	Tutor search and filter logic
TutorHub.Application/Services/	NotificationService.cs	Email and in-app notifications
TutorHub.Application/Services/	AdminService.cs	Admin management logic
TutorHub.Application/Services/	EarningsService.cs	Earnings and payout calculations
TutorHub.Application/Validators/	RegisterValidator.cs	FluentValidation for registration
TutorHub.Application/Validators/	BookingValidator.cs	Booking business rule validation
TutorHub.Application/Validators/	ReviewValidator.cs	Review validation rules
TutorHub.Application/Mappings/	MappingProfile.cs	AutoMapper configuration
TutorHub.Application/Helpers/	PaginationHelper.cs	Paginated list helper
TutorHub.Application/Helpers/	DateTimeHelper.cs	Time zone and date utilities

TutorHub.Infrastructure (Data + External Services Layer)

Folder Path	File Name	Purpose
TutorHub.Infrastructure/Data/	TutorHubDbContext.cs	EF Core DbContext — main entry point
TutorHub.Infrastructure/Data/Configurations/	TutorProfileConfiguration.cs	EF Core Fluent API config
TutorHub.Infrastructure/Data/Configurations/	BookingConfiguration.cs	EF Core Fluent API config
TutorHub.Infrastructure/Data/Configurations/	PaymentConfiguration.cs	EF Core Fluent API config
TutorHub.Infrastructure/Data/Configurations/	ReviewConfiguration.cs	EF Core Fluent API config

TutorHub.Infrastructure/Data/Seed/	DataSeeder.cs	Seeds subjects, grade levels, admin user
TutorHub.Infrastructure/Repositories/	UserRepository.cs	IUserRepository implementation
TutorHub.Infrastructure/Repositories/	TutorRepository.cs	ITutorRepository implementation
TutorHub.Infrastructure/Repositories/	BookingRepository.cs	IBookingRepository implementation
TutorHub.Infrastructure/Repositories/	PaymentRepository.cs	IPaymentRepository implementation
TutorHub.Infrastructure/Repositories/	ReviewRepository.cs	IReviewRepository implementation
TutorHub.Infrastructure/Repositories/	ChatRepository.cs	IChatRepository implementation
TutorHub.Infrastructure/ExternalServices/	EmailService.cs	SendGrid email sending
TutorHub.Infrastructure/ExternalServices/	SmsService.cs	Twilio SMS sending
TutorHub.Infrastructure/ExternalServices/	FileStorageService.cs	Azure Blob Storage upload
TutorHub.Infrastructure/ExternalServices/	StripePaymentService.cs	Stripe payment processing
TutorHub.Infrastructure/Security/	TokenService.cs	JWT generation and refresh tokens
TutorHub.Infrastructure/Security/	PasswordHasher.cs	BCrypt password hashing
TutorHub.Infrastructure/Migrations/	(auto-generated)	EF Core database migrations
TutorHub.Infrastructure/DependencyInjection.cs		Registers all services in DI container

TutorHub.API (Presentation Layer — Controllers)

File Name	Controller Class	Purpose
Controllers/AuthController.cs	AuthController	Registration, login, password management
Controllers/TutorController.cs	TutorController	Tutor profile and availability
Controllers/StudentController.cs	StudentController	Student profile and bookings
Controllers/BookingController.cs	BookingController	Booking CRUD and lifecycle
Controllers/PaymentController.cs	PaymentController	Stripe payment flows
Controllers/ReviewController.cs	ReviewController	Reviews and ratings
Controllers/ChatController.cs	ChatController	Chat history (REST) + SignalR
Controllers/SearchController.cs	SearchController	Tutor search and filters
Controllers/NotificationController.cs	NotificationController	Notification management

Controllers/AdminController.cs	AdminController	Full admin operations
Hubs/ChatHub.cs	ChatHub (SignalR)	Real-time WebSocket chat
Middleware/ExceptionMiddleware.cs		Global exception handling
Middleware/RateLimitMiddleware.cs		100 requests/minute per IP
Program.cs		App startup, DI configuration, middleware pipeline
appsettings.json		App configuration (no secrets in this file)
appsettings.Development.json		Local dev overrides
.env / User Secrets		JWT secret, DB connection, Stripe keys (NEVER commit to Git)

1.4 NuGet Packages to Install

In Visual Studio, open the Package Manager Console and run each command in the TutorHub.API project unless otherwise noted.

Package Name	Project Target	Install Command
Microsoft.AspNetCore.Authentication.JwtBearer	TutorHub.API	dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
Microsoft.AspNetCore.Identity.EntityFrameworkCore	TutorHub.Infrastructure	dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore
Microsoft.EntityFrameworkCore.SqlServer	TutorHub.Infrastructure	dotnet add package Microsoft.EntityFrameworkCore.SqlServer
Microsoft.EntityFrameworkCore.Tools	TutorHub.Infrastructure	dotnet add package Microsoft.EntityFrameworkCore.Tools
AutoMapper.Extensions.Microsoft.DependencyInjection	TutorHub.Application	dotnet add package AutoMapper.Extensions.Microsoft.DependencyInjection
FluentValidation.AspNetCore	TutorHub.Application	dotnet add package FluentValidation.AspNetCore
BCrypt.Net-Next	TutorHub.Infrastructure	dotnet add package BCrypt.Net-Next
Swashbuckle.AspNetCore	TutorHub.API	dotnet add package Swashbuckle.AspNetCore
Microsoft.AspNetCore.SignalR	TutorHub.API	dotnet add package Microsoft.AspNetCore.SignalR
SendGrid	TutorHub.Infrastructure	dotnet add package SendGrid
Twilio	TutorHub.Infrastructure	dotnet add package Twilio
Azure.Storage.Blobs	TutorHub.Infrastructure	dotnet add package Azure.Storage.Blobs

Stripe.net	TutorHub.Infrastructure	dotnet add package Stripe.net
Serilog.AspNetCore	TutorHub.API	dotnet add package Serilog.AspNetCore
AspNetCoreRateLimit	TutorHub.API	dotnet add package AspNetCoreRateLimit
StackExchange.Redis	TutorHub.Infrastructure	dotnet add package StackExchange.Redis
ClosedXML	TutorHub.Infrastructure	dotnet add package ClosedXML (for Excel export)
iTextSharp	TutorHub.Infrastructure	dotnet add package iTextSharp (for PDF export)

1.5 JWT Security Implementation

The JWT system is entirely contained in two files. This is where all authentication security lives.

File: TutorHub.Infrastructure/Security/TokenService.cs

- This class implements `ITokenService` from the Core layer.
- Method `GenerateAccessToken(ApplicationUser user)` — Creates a JWT with these claims: `UserId (user.Id)`, `Email (user.Email)`, `Role (user.Role.ToString())`, and standard claims (`jti`, `iat`, `exp`). Token expiry is 60 minutes. Sign using `HmacSha256` algorithm.
- Method `GenerateRefreshToken()` — Creates a random 64-byte Base64 string. This is stored in the `RefreshToken` entity in the database.
- Method `GetPrincipalFromExpiredToken(string token)` — Validates an expired access token to extract claims for the refresh flow.

File: Program.cs — JWT Configuration Block

- Add `builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)` section.
- Configure `ValidateIssuer = true`, `ValidateAudience = true`, `ValidateLifetime = true`, `ValidateIssuerSigningKey = true`.
- `Issuer = configuration['Jwt:Issuer']`, `Audience = configuration['Jwt:Audience']`.
- `IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(configuration['Jwt:SecretKey']))`.

File: appsettings.json — JWT Section:

- Add a "Jwt" section with keys: `Issuer = "TutorHub"`, `Audience = "TutorHubUsers"`, `ExpiryMinutes = 60`.
- IMPORTANT: The actual `SecretKey` value must be stored in User Secrets (local) or Azure Key Vault (production). Never put it in `appsettings.json`.

File: All Controllers — Authorization Attributes:

- `[Authorize]` — any logged-in user.
- `[Authorize(Roles = "Admin")]` — admin only endpoints.
- `[Authorize(Roles = "Tutor")]` — tutor-only endpoints.
- `[Authorize(Roles = "Student,Parent")]` — booking and payment endpoints.

1.6 Complete API Endpoint Reference

All 94+ endpoints organized by controller. Haseeb must build each endpoint with proper HTTP verb, route, authorization, and response codes.

AuthController — Base Route: /api/auth

HTTP Method + Endpoint	Authorization	Description
POST /api/auth/register	Public	Register new user (Student/Parent/Tutor)
POST /api/auth/login	Public	Login with email and password, returns JWT
POST /api/auth/refresh-token	Public	Exchange refresh token for new access token
POST /api/auth/logout	Authenticated	Revoke refresh token
POST /api/auth/verify-email	Public	Verify email with token from email link
POST /api/auth/forgot-password	Public	Send password reset email
POST /api/auth/reset-password	Public	Reset password with token
PUT /api/auth/change-password	Authenticated	Change password while logged in

TutorController — Base Route: /api/tutor

HTTP Method + Endpoint	Authorization	Description
GET /api/tutor/{id}	Public	Get tutor public profile by user ID
PUT /api/tutor/profile	Tutor	Update own tutor profile
POST /api/tutor/documents	Tutor	Upload verification documents (multipart/form-data)
GET /api/tutor/documents	Tutor	View own uploaded documents and status
GET /api/tutor/availability	Tutor	Get own weekly availability
POST /api/tutor/availability	Tutor	Set weekly recurring availability
PUT /api/tutor/availability/{id}	Tutor	Update specific availability slot
DELETE /api/tutor/availability/{id}	Tutor	Delete availability slot
GET /api/tutor/schedule	Tutor	View calendar with bookings and availability
GET /api/tutor/bookings	Tutor	View all bookings (upcoming and past)
GET /api/tutor/earnings	Tutor	Earnings summary with period filter
GET /api/tutor/earnings/report	Tutor	Download earnings report (CSV or PDF)
GET /api/tutor/reviews	Tutor	View own reviews and ratings
POST /api/tutor/payout/request	Tutor	Request manual payout
GET /api/tutor/payout/history	Tutor	View payout history

BookingController — Base Route: /api/booking

HTTP Method + Endpoint	Authorization	Description
POST /api/booking	Student / Parent	Create new booking request
GET /api/booking/{id}	Authenticated	Get booking details by ID
GET /api/booking/my	Authenticated	Get all bookings for logged-in user
PUT /api/booking/{id}/confirm	Tutor	Tutor confirms a pending booking
PUT /api/booking/{id}/cancel	Authenticated	Cancel a booking with reason
PUT /api/booking/{id}/reschedule	Authenticated	Request reschedule (tutor must approve)

PUT /api/booking/{id}/complete	Tutor	Mark session as completed
GET /api/booking/{id}/meeting-link	Authenticated	Get video call link for session
POST /api/booking/{id}/report	Student / Parent	Report an issue with a session

PaymentController — Base Route: /api/payment

HTTP Method + Endpoint	Authorization	Description
POST /api/payment/create-intent	Student / Parent	Create Stripe PaymentIntent for booking
POST /api/payment/confirm	Student / Parent	Confirm payment after Stripe checkout
GET /api/payment/history	Authenticated	Get payment transaction history
GET /api/payment/{id}	Authenticated	Get specific payment details
POST /api/payment/refund/{bookingId}	Admin	Process manual refund
POST /api/payment/webhook	Public (Stripe only)	Stripe webhook endpoint for events

SearchController — Base Route: /api/search

HTTP Method + Endpoint	Authorization	Description
GET /api/search/tutors	Public	Search tutors with filters (query params: keyword, subject, gradeLevel, minPrice, maxPrice, rating, teachingMode, city, page, pageSize)
GET /api/search/autocomplete	Public	Return autocomplete suggestions for search bar
GET /api/search/subjects	Public	List all available subjects
GET /api/search/grade-levels	Public	List all grade levels

ReviewController — Base Route: /api/review

HTTP Method + Endpoint	Authorization	Description
POST /api/review	Student / Parent	Submit review after completed session
GET /api/review/tutor/{tutorId}	Public	Get all reviews for a tutor
PUT /api/review/{id}	Student / Parent	Edit own review (within 7 days)
DELETE /api/review/{id}	Student / Parent / Admin	Delete review

ChatController + ChatHub — SignalR Real-Time Chat

Endpoint / Method	Type	Description
GET /api/chat/conversations	REST — Authenticated	Get all conversations for current user
GET /api/chat/messages/{conversationId}	REST — Authenticated	Get chat history for a conversation

POST /api/chat/conversations	REST — Authenticated	Start new conversation with a user
ChatHub — Hub URL: /hubs/chat	SignalR	WebSocket connection endpoint
ChatHub.SendMessage(conversationId, message)	SignalR Client → Server	Client calls to send message
ChatHub.MarkAsRead(conversationId)	SignalR Client → Server	Client marks messages as read
ChatHub.ReceiveMessage(message)	SignalR Server → Client	Server pushes new message to client
ChatHub.MessageRead(conversationId)	SignalR Server → Client	Server notifies read receipt
ChatHub.UserOnline(userId)	SignalR Server → Client	Notifies when user goes online

AdminController — Base Route: /api/admin

HTTP Method + Endpoint	Authorization	Description
GET /api/admin/users	Admin	List all users with filters
GET /api/admin/users/{id}	Admin	Get specific user details
PUT /api/admin/users/{id}/suspend	Admin	Suspend a user account
PUT /api/admin/users/{id}/activate	Admin	Re-activate a user account
DELETE /api/admin/users/{id}	Admin	Delete user (GDPR)
GET /api/admin/tutor-applications	Admin	List pending tutor approvals
GET /api/admin/tutor-applications/{id}	Admin	View tutor with documents
PUT /api/admin/tutor-applications/{id}/approve	Admin	Approve tutor
PUT /api/admin/tutor-applications/{id}/reject	Admin	Reject tutor with reason
GET /api/admin/bookings	Admin	View all platform bookings
PUT /api/admin/bookings/{id}/cancel	Admin	Cancel any booking
POST /api/admin/refunds/{paymentId}	Admin	Manual refund override
GET /api/admin/analytics/dashboard	Admin	Key metrics summary
GET /api/admin/analytics/revenue	Admin	Revenue report by period
GET /api/admin/analytics/users	Admin	User growth analytics
GET /api/admin/subjects	Admin	List all subjects
POST /api/admin/subjects	Admin	Add new subject

DELETE /api/admin/subjects/{id}	Admin	Remove subject
---------------------------------	-------	----------------

1.7 Database Setup and EF Core Migrations

After writing all entity classes, run these commands in this exact order to set up the database.

1. Open appsettings.Development.json and add the ConnectionStrings section: DefaultConnection = 'Server=(localdb)\mssqllocaldb;Database=TutorHubDb;Trusted_Connection=True'
2. In TutorHubDbContext.cs, inherit from IdentityDbContext<ApplicationUser> and add DbSet<T> properties for every entity class listed in Section 1.3.
3. In TutorHub.API Package Manager Console: dotnet ef migrations add InitialCreate --project TutorHub.Infrastructure --startup-project TutorHub.API
4. Apply migration: dotnet ef database update --project TutorHub.Infrastructure --startup-project TutorHub.API
5. Run DataSeeder.cs on startup to seed: 20 subjects (Mathematics, Physics, Chemistry, English, Biology, Computer Science, Urdu, Islamiyat, History, Geography, Economics, Accounting, Business Studies, Psychology, Sociology, Statistics, Art, Music, Physical Education, Arabic), 8 grade levels (Primary 1-5, Secondary 6-10, O-Level, A-Level, University, Other), and one Admin user.
6. Every time you add a new property to an entity: dotnet ef migrations add [MigrationName] and then dotnet ef database update.

BACKEND WEEKLY SCHEDULE

Haseeb — 2 Hours Daily | April 20 — May 20, 2026

Section 2 — Backend 30-Day Daily Schedule

Week 1: Apr 20–26 — Foundation, Auth & Database

Goal: Working API that can register users, log in, issue JWT tokens, and read/write to the database.

WEEK 1	Haseeb — ASP.NET Core Backend	Talha — React Frontend
Mon Apr 20	Create solution and all 4 projects (API, Core, Application, Infrastructure). Add all project references. Install all NuGet packages. Push initial commit to Git.	
Tue Apr 21	Create all Core entities (ApplicationUser, TutorProfile, StudentProfile, RefreshToken). Create all Core enums (UserRole, BookingStatus, TeachingMode, DocumentStatus). Create all Core interfaces.	
Wed Apr 22	Set up TutorHubDbContext.cs with IdentityDbContext. Configure EF Core Fluent API for ApplicationUser. Add connection string to appsettings. Run first migration (InitialCreate). Confirm database created in SSMS.	
Thu Apr 23	Implement TokenService.cs — JWT access token generation and refresh token generation. Configure JWT middleware in Program.cs. Add User Secrets for JWT secret key.	
Fri Apr 24	Implement AuthService.cs — Register method (hash password, save user, send verification email via EmailService stub). Implement Login method (validate credentials, check logout, return tokens). Store refresh token in database.	
Sat Apr 25	Create AuthController.cs with POST /api/auth/register, POST /api/auth/login, POST /api/auth/refresh-token. Test all 3 endpoints in Postman. Verify JWT token can be decoded at jwt.io and contains correct claims.	
Sun Apr 26	Add forgot-password and reset-password endpoints. Implement email verification flow. Add rate limiting for auth endpoints (5 failed attempts = 15 min logout using	

ASP.NET Core Identity lockout settings).
Weekly sync with Talha — share API base URL and Swagger docs.

Week 2: Apr 27 — May 3 — Tutor Module & Search

Goal: Tutors can complete profiles, upload documents, set availability. Search endpoint returns paginated tutor results.

WEEK 2	Haseeb — ASP.NET Core Backend	Talha — React Frontend
Mon Apr 27	Create remaining Core entities: TutorAvailability, TutorDocument, TutorSubject, Subject, GradeLevel. Run migration AddTutorEntities. Seed subjects and grade levels via DataSeeder.cs.	
Tue Apr 28	Implement TutorService.cs — GetTutorProfile, UpdateTutorProfile, UploadDocument (Azure Blob Storage integration). Test document upload to Azure Blob — get storage connection string from Azure portal.	
Wed Apr 29	Implement TutorService.cs — SetAvailability (save weekly slots), GetAvailability, BlockTimeSlot. Add duplicate-slot prevention logic in BookingRepository. Create TutorController.cs with profile, document, availability endpoints.	
Thu Apr 30	Build SearchService.cs — query IQueryable<TutorProfile> with filters: keyword (search Headline + Bio + subjects), subjectId, gradeLevelId, minPrice, maxPrice, minRating, teachingMode, city. Add sorting logic (by rating, price, experience). Add pagination using PaginationHelper.cs.	
Fri May 1	Create SearchController.cs with GET /api/search/tutors. Add GET /api/search/autocomplete that returns matching tutor names and subject names. Add GET /api/search/subjects and /grade-levels. Test all search scenarios in Postman.	
Sat May 2	Implement TutorDashboard data — upcoming bookings list, past bookings history, earnings summary. Create TutorEarningsDto with daily/weekly/monthly breakdown. Add /api/tutor/earnings endpoint.	
Sun May 3	Weekly integration sync with Talha. Provide Talha with: complete Swagger JSON URL, authentication sample requests, and sample tutor search response JSON. Fix	

any CORS issues preventing Talha's React app from connecting.

Week 3: May 4–10 — Booking, Payment & Reviews

Goal: End-to-end booking flow works. Stripe payments are processed. Reviews can be submitted after completed sessions.

WEEK 3	Haseeb — ASP.NET Core Backend	Talha — React Frontend
Mon May 4	Create Booking, Payment, Review, Payout entities. Run migration AddBookingPaymentReview. Implement BookingRepository.cs with methods: CreateBooking, GetBookingById, GetBookingsByUser, UpdateBookingStatus.	
Tue May 5	Implement BookingService.cs — CreateBooking (validate tutor availability, check no double-booking, calculate amounts: tutor fee, platform fee 10%, total). Implement ConfirmBooking (tutor accepts, update availability to blocked).	
Wed May 6	Add CancelBooking logic in BookingService — apply refund rules (100% if 24h+, 50% if 12-24h, 0% if under 12h). Add auto-cancel logic for unanswered requests after 24 hours (use background service or Hangfire). Create BookingController.cs.	
Thu May 7	Set up Stripe integration in StripePaymentService.cs — Initialize with Stripe secret key (from user secrets). Implement CreatePaymentIntent method (create Stripe PaymentIntent, store in Payment table). Implement ConfirmPayment after client confirms.	
Fri May 8	Add Stripe Webhook endpoint POST /api/payment/webhook — handle events: payment_intent.succeeded (confirm booking), payment_intent.payment_failed (cancel booking), charge.refunded (update refund status). Test with Stripe CLI locally: stripe listen --forward-to localhost:5000/api/payment/webhook.	
Sat May 9	Implement ReviewService.cs — SubmitReview (validate booking is completed, validate 14-day window, validate one review per booking). UpdateTutorAverageRating after each review (recalculate from all reviews). Create ReviewController.cs.	
Sun May 10	Weekly sync with Talha. Share booking flow sequence diagram. Confirm Stripe test	

keys work from React (Talha uses stripe.js publishable key, Haseeb uses secret key on backend). Test end-to-end: React creates PaymentIntent → Stripe.js confirms → Webhook fires → Booking confirmed.

Week 4: May 11–17 — Chat, Admin & Notifications

Goal: Real-time chat works via SignalR. Admin panel is complete. Email notifications are firing.

WEEK 4	Haseeb — ASP.NET Core Backend	Talha — React Frontend
Mon May 11	Create ChatConversation and ChatMessage entities. Run migration AddChat. Implement ChatRepository.cs — GetConversations (for current user), GetMessages (by conversationId, paginated), CreateMessage.	
Tue May 12	Build ChatHub.cs (SignalR Hub) — override OnConnectedAsync (add user to their personal group using userId). Implement SendMessage server method (save to DB, push to recipient group via Clients.Group). Implement MarkAsRead. Add connection management for online/offline status.	
Wed May 13	Implement NotificationService.cs — SendEmailNotification using SendGrid API. Create email templates for: Welcome, EmailVerification, BookingConfirmed, BookingCancelled, BookingReminder (1 hour before), PaymentReceipt, TutorApproved. Store in-app notifications in Notification table.	
Thu May 14	Build AdminService.cs — GetAllUsers (with filters), SuspendUser, ActivateUser, DeleteUser (anonymize for GDPR). GetPendingTutorApplications. ApproveTutor (set IsVerified=true, send approval email). RejectTutor (send rejection email with reason).	
Fri May 15	Build AdminController.cs with all 18 admin endpoints. Add platform analytics endpoint — GetDashboardStats: total users count by role, bookings today/this week, revenue today/this week, pending tutor applications count. Add all admin routes protected with [Authorize(Roles='Admin')] attribute.	
Sat May 16	Add EarningsService.cs — calculate tutor earnings (group completed bookings by day/week/month). Add payout request logic (minimum \$50 threshold check). Add weekly automatic payout trigger (can simulate with manual endpoint for now). Generate earnings CSV using ClosedXML.	

Sun May 17	Full backend integration test day. Use Postman collection to run all 94+ endpoints. Fix bugs. Confirm SignalR chat works (test in browser console). Confirm all email notifications fire correctly using SendGrid sandbox mode. Sync with Talha for major integration test session.	
-------------------	---	--

Week 5: May 18–20 — Backend Polish, Security & Pre-Deployment

Goal: Backend is production-ready, secure, documented, and ready for Azure deployment.

WEEK 5	Haseeb — ASP.NET Core Backend	Talha — React Frontend
Mon May 18	Security hardening: Confirm CORS only allows Talha's React domain. Add global exception middleware (ExceptionHandler.cs) that returns standardized error responses. Add request logging with Serilog. Confirm rate limiting works (test with 100+ rapid requests). Add input validation to all DTOs using FluentValidation.	
Tue May 19	Configure Swagger for production — add JWT authorization button to Swagger UI. Document all endpoints with XML comments. Add API versioning prefix (/api/v1/). Create Postman collection JSON and share with Talha. Configure HTTPS redirection.	
Wed May 20	Final backend review: Check all endpoints return consistent response format (statusCode, data, message, errors). Check all sensitive data is not exposed in responses. Run dotnet publish in Release mode. Confirm no build warnings.	

PART B — FRONTEND

Talha | React 18 + Vite | 2 Hours Daily

Section 3 — Frontend Project Setup & Architecture

3.1 Tools and Software to Install

Tool	Version	Purpose
Node.js	20 LTS	Required runtime for React + Vite
npm	Comes with Node.js	Package manager
VS Code	Latest	Main IDE for frontend
VS Code Extensions	ES7 React Snippets, Tailwind CSS IntelliSense, Prettier, GitLens	Developer productivity
Git	Latest	Version control
Chrome DevTools	Built-in	Debugging
React Developer Tools	Chrome Extension	React debugging

3.2 Create React Project with Vite

Step 1 — Scaffold the project:

- Open terminal in a new folder (NOT inside Haseeb's backend folder).
- Command: `npm create vite@latest tutorhub-frontend -- --template react`
- Command: `cd tutorhub-frontend`
- Command: `npm install`
- Command: `npm run dev` (confirms Vite dev server runs on localhost:5173)

Step 2 — Install all required npm packages:

- Command: `npm install react-router-dom axios @tanstack/react-query zustand react-hook-form @hookform/resolvers zod`
- Command: `npm install @stripe/stripe-js @stripe/react-stripe-js`
- Command: `npm install @microsoft/signalr`
- Command: `npm install react-hot-toast react-datepicker react-select react-star-ratings`
- Command: `npm install date-fns recharts`
- Command: `npm install -D tailwindcss postcss autoprefixer`
- Command: `npx tailwindcss init -p`
- Command: `npm install -D @types/node`

3.3 Package Purpose Reference

Package Name	Purpose	Used In
react-router-dom	Page routing and navigation	App.jsx — all route definitions
axios	HTTP calls to Haseeb's backend API	src/services/ — all API files
@tanstack/react-query	Server state management, caching, loading/error states	All pages that fetch data
zustand	Client-side global state (auth user, notifications)	src/store/ — authStore.js
react-hook-form	Form state management and validation	All form components
zod	Schema-based form validation rules	src/validators/ — all validation schemas
@stripe/stripe-js + react-stripe-js	Stripe payment UI (card element)	PaymentPage.jsx
@microsoft/signalr	Real-time chat WebSocket connection	ChatPage.jsx, chatService.js
react-hot-toast	Success/error notification toasts	Everywhere after API calls
react-datepicker	Date selection for booking form	BookSessionPage.jsx
react-select	Multi-select for subjects and filters	SearchPage.jsx, TutorProfileForm.jsx
date-fns	Date formatting utilities	Booking cards, chat timestamps
recharts	Charts for admin and tutor dashboards	AdminDashboard.jsx, EarningsPage.jsx
tailwindcss	Utility-first CSS styling	All components

3.4 Complete Folder and File Directory Structure

Every folder and file in the React project is listed below. Create this structure before writing any component logic.

Root Configuration Files

File Name	Purpose	Notes
vite.config.js	Vite build config — add proxy for API	Proxy /api to Haseeb's localhost:5000
.env	Environment variables (never commit)	VITE_API_URL, VITE_STRIPE_PK
.env.production	Production environment values	VITE_API_URL = Azure URL
tailwind.config.js	Tailwind configuration with custom colors	Add TutorHub brand colors
index.html	Root HTML file	Auto-generated by Vite
src/main.jsx	React entry point — mount App	Wrap with QueryClientProvider, Router
src/App.jsx	Root component — all route definitions	Contains all React Router routes

src/index.css	Global styles and Tailwind imports	Add @tailwind base/components/utilities
---------------	------------------------------------	--

src/services/ — API Layer (All Axios Calls)

File Name	Functions Inside	Calls to Backend
axiosInstance.js	Create axios instance with baseURL, interceptors for JWT header, interceptor for 401 to refresh token	All API calls use this instance
authService.js	register(data), login(data), refreshToken(), logout(), forgotPassword(email), resetPassword(token, newPassword), verifyEmail(token)	/api/auth/*
tutorService.js	getTutorProfile(id), updateTutorProfile(data), uploadDocument(file, type), getAvailability(), setAvailability(slots), getTutorBookings(), getEarnings(period), requestPayout()	/api/tutor/*
bookingService.js	createBooking(data), getBooking(id), getMyBookings(), confirmBooking(id), cancelBooking(id, reason), rescheduleBooking(id, data), completeBooking(id)	/api/booking/*
paymentService.js	createPaymentIntent(bookingId), confirmPayment(paymentIntentId), getPaymentHistory()	/api/payment/*
searchService.js	searchTutors(filters), getAutocomplete(query), getSubjects(), getGradeLevels()	/api/search/*
reviewService.js	submitReview(data), getTutorReviews(tutorId), editReview(id, data), deleteReview(id)	/api/review/*
chatService.js	getConversations(), getMessages(conversationId), startConversation(userId). Also: connectSignalR(), disconnectSignalR(), sendMessage(convId, msg), onMessageReceived(callback)	/api/chat/* + SignalR hub
notificationService.js	getNotifications(), markAsRead(id), markAllRead()	/api/notification/*
adminService.js	getUsers(filters), suspendUser(id), activateUser(id), deleteUser(id), getTutorApplications(), approveTutor(id), rejectTutor(id, reason), getDashboardStats(), getRevenue(period)	/api/admin/*
studentService.js	getStudentProfile(), updateStudentProfile(data), getChildProfiles(), addChildProfile(data)	/api/student/*

src/store/ — Global State (Zustand)

File Name	State Variables	Actions
authStore.js	user (object), token (string), isAuthenticated (bool), isLoading (bool)	login(userData, token), logout(), updateUser(data), setLoading(bool)
notificationStore.js	notifications (array), unreadCount (number)	addNotification(n), markRead(id), clearAll()
chatStore.js	conversations (array), activeConversation (object), messages (array), isConnected (bool)	setConversations(list), setMessages(msgs), addMessage(msg), setConnected(bool)

src/pages/ — All Application Pages

File Name	Route Path	What This Page Does
HomePage.jsx	/	Landing page: hero section, search bar, how it works, featured tutors
SearchPage.jsx	/search	Search results with filters sidebar, tutor cards, pagination
TutorProfilePage.jsx	/tutor/:id	Full tutor public profile: bio, subjects, availability calendar, reviews, book session form
BookSessionPage.jsx	/book/:tutorId	Multi-step booking form: select subject, date, time, duration, mode, review price
PaymentPage.jsx	/payment/:bookingId	Stripe card element, payment breakdown summary, submit payment
PaymentSuccessPage.jsx	/payment/success	Booking confirmation page with details
LoginPage.jsx	/login	Email + password login form
RegisterPage.jsx	/register	Registration form with role selection
ForgotPasswordPage.jsx	/forgot-password	Email input for reset link
ResetPasswordPage.jsx	/reset-password	New password form with token
VerifyEmailPage.jsx	/verify-email	Email verification landing page
StudentDashboard.jsx	/dashboard/student	Upcoming bookings, quick actions, recent messages
MyBookingsPage.jsx	/dashboard/bookings	All bookings list with tabs: Upcoming, Past, Cancelled
PaymentHistoryPage.jsx	/dashboard/payments	Payment transaction history
ParentDashboard.jsx	/dashboard/parent	Children overview, all children bookings, spending summary
ChildProfilesPage.jsx	/dashboard/children	Manage child profiles (add, edit)
TutorDashboard.jsx	/dashboard/tutor	Upcoming sessions, earnings summary, pending approvals count

TutorProfileEditPage.jsx	/dashboard/tutor/profile	Edit tutor profile: headline, bio, education, subjects, rate, mode
TutorAvailabilityPage.jsx	/dashboard/tutor/availability	Weekly availability grid — click slots to set/unset
TutorDocumentsPage.jsx	/dashboard/tutor/documents	Upload government ID and certificates, view approval status
TutorBookingsPage.jsx	/dashboard/tutor/bookings	Tutor view of all sessions — confirm/cancel pending requests
TutorEarningsPage.jsx	/dashboard/tutor/earnings	Earnings charts (recharts), payout history, request payout button
ChatPage.jsx	/chat	Real-time chat: conversations list sidebar, active chat window, message input
NotificationsPage.jsx	/notifications	All notifications with read/unread status
ProfileSettingsPage.jsx	/settings	Edit personal info, change password, notification preferences
AdminDashboard.jsx	/admin	Platform metrics cards, charts: bookings over time, top tutors, revenue
AdminUsersPage.jsx	/admin/users	User management table with search, filters, suspend/activate
AdminTutorApprovalsPage.jsx	/admin/approvals	Pending tutor applications with document viewer
AdminBookingsPage.jsx	/admin/bookings	All platform bookings with cancel and refund actions
AdminAnalyticsPage.jsx	/admin/analytics	Advanced reporting charts and export buttons
AdminSubjectsPage.jsx	/admin/subjects	CRUD for subjects and grade levels
NotFoundPage.jsx	*	404 page not found

src/components/ — Reusable Components

File Name	What It Renders	Used In
layout/Navbar.jsx	Top navigation bar with logo, nav links, login/register or user menu dropdown	All pages
layout/Footer.jsx	Footer with links and copyright	All public pages
layout/Sidebar.jsx	Dashboard sidebar navigation (different links per role)	All dashboard pages
layout/DashboardLayout.jsx	Wrapper: Sidebar + main content area + Navbar	All dashboard pages
layout/ProtectedRoute.jsx	Redirects unauthenticated users to /login. Also checks role permissions.	App.jsx routes
tutor/TutorCard.jsx	Search result card: avatar, name, rating, subjects, price, location, View Profile button	SearchPage.jsx

tutor/TutorAvailabilityGrid.jsx	Weekly grid showing available/booked slots, clickable for booking	TutorProfilePage.jsx
tutor/TutorReviewList.jsx	List of reviews with star rating, reviewer name, date, comment	TutorProfilePage.jsx
tutor/TutorBadge.jsx	Verified badge icon shown on verified tutors	TutorCard, TutorProfilePage
booking/BookingCard.jsx	Single booking card: tutor name, subject, date, time, status badge, action buttons	MyBookingsPage, dashboards
booking/BookingStatusBadge.jsx	Colored badge: Pending/Confirmed/Cancelled/Completed	BookingCard.jsx
booking/CancelBookingModal.jsx	Modal with reason input and confirm button for cancellation	BookingCard.jsx
search/SearchBar.jsx	Main search input with autocomplete dropdown suggestions	HomePage.jsx, SearchPage.jsx
search/FilterPanel.jsx	Sidebar filters: price slider, rating checkboxes, subject multi-select, teaching mode, city	SearchPage.jsx
search/SortDropdown.jsx	Sort by: Rating, Price Low-High, Price High-Low, Experience, Most Reviewed	SearchPage.jsx
payment/StripeCardElement.jsx	Stripe Elements card input wrapped in React component	PaymentPage.jsx
payment/PriceBreakdown.jsx	Shows: Session fee, Platform fee (10%), Total	PaymentPage.jsx, BookSessionPage
chat/ConversationList.jsx	List of all chat conversations with last message preview and unread count badge	ChatPage.jsx
chat/ChatWindow.jsx	Active chat: message bubbles, timestamps, read receipts, typing indicator	ChatPage.jsx
chat/MessageInput.jsx	Text input with send button and file attach button	ChatPage.jsx
admin/MetricCard.jsx	Stat card: icon, label, value, trend arrow	AdminDashboard.jsx
admin/UserTableRow.jsx	Single row in user management table	AdminUsersPage.jsx
admin/TutorApplicationCard.jsx	Card showing tutor application with document links, approve/reject buttons	AdminTutorApprovalsPage.jsx
common/StarRating.jsx	Clickable or display-only star rating (1-5)	Review forms, TutorCard
common/Pagination.jsx	Page number navigation with Prev/Next buttons	SearchPage, AdminUsersPage
common/Modal.jsx	Reusable modal wrapper with overlay and close button	Throughout app
common/LoadingSpinner.jsx	Centered spinning animation for loading states	Throughout app
common/ErrorMessage.jsx	Styled error display for API errors	Throughout app
common/EmptyState.jsx	Empty state illustration with message and CTA button	Empty lists throughout app

common/ConfirmDialog.jsx	Generic confirmation modal: Are you sure? Yes/Cancel	Delete and cancel actions
common/FileUpload.jsx	Drag-and-drop file upload with preview and size validation	TutorDocumentsPage.jsx
common/NotificationBell.jsx	Bell icon with unread count badge in Navbar	Navbar.jsx
common/Avatar.jsx	User avatar: profile image or initials fallback	Navbar, chat, tutor cards
forms/LoginForm.jsx	Email + password form with remember me checkbox	LoginPage.jsx
forms/RegisterForm.jsx	Full registration form with role toggle	RegisterPage.jsx
forms/ReviewForm.jsx	Star rating + comment textarea + submit button	TutorProfilePage, after session
forms/ChildProfileForm.jsx	Add/edit child: name, grade level, subjects	ChildProfilesPage.jsx

src/validators/ — Zod Validation Schemas

File Name	Validates	Key Rules
authValidators.js	Login and Register forms	Email format required, Password min 8 chars with uppercase + number + special char
bookingValidators.js	Booking form	Date must be 2+ hours in future, Duration must be 30/60/90/120 only
reviewValidators.js	Review form	Rating required (1-5), Comment max 500 chars
tutorProfileValidators.js	Tutor profile edit form	HourlyRateMin must be less than HourlyRateMax, Bio min 50 chars
passwordValidators.js	Change/reset password	New password matches confirm password

src/hooks/ — Custom React Hooks

File Name	Purpose	Returns
useAuth.js	Access auth state and actions from Zustand store	user, isAuthenticated, login(), logout()
useTutorSearch.js	Manages search filters state and calls searchService	tutors, isLoading, error, filters, setFilter, resetFilters
useBookingForm.js	Multi-step booking form state management	step, formData, nextStep(), prevStep(), setField()
useSignalR.js	Connect and manage SignalR chat connection lifecycle	isConnected, sendMessage(convId, msg), disconnect()
useNotifications.js	Fetch and manage notifications with polling	notifications, unreadCount, markRead()
usePagination.js	Handles page number state for paginated lists	currentPage, totalPages, goToPage(), nextPage(), prevPage()

src/utils/ — Utility Functions

File Name	Functions	Purpose
formatters.js	formatCurrency(amount), formatDate(date), formatTime(time), formatDuration(mins)	Display formatting helpers
tokenUtils.js	getTokenFromStorage(), saveToken(token), clearToken(), isTokenExpired(token)	JWT token management in localStorage
roleUtils.js	isAdmin(user), isTutor(user), isStudent(user), isParent(user)	Role check helpers for conditional rendering
errorUtils.js	parseApiError(error)	Extracts readable message from axios error response
constants.js	BOOKING_STATUSES, TEACHING_MODES, DURATION_OPTIONS, PLATFORM_FEE_PERCENT	App-wide constants

3.5 React Router Setup in App.jsx

All routes are defined in App.jsx. Protected routes use ProtectedRoute.jsx to check authentication and role.

Public routes (no login required): /, /search, /tutor/:id, /login, /register, /forgot-password, /reset-password, /verify-email.

Authenticated routes (any logged-in user): /chat, /notifications, /settings, /dashboard/bookings, /payment/:bookingId, /payment/success, /dashboard/payments.

Student / Parent only routes: /book/:tutorId, /dashboard/student, /dashboard/parent, /dashboard/children.

Tutor only routes: /dashboard/tutor, /dashboard/tutor/profile, /dashboard/tutor/availability, /dashboard/tutor/documents, /dashboard/tutor/bookings, /dashboard/tutor/earnings.

Admin only routes: /admin, /admin/users, /admin/approvals, /admin/bookings, /admin/analytics, /admin/subjects.

3.6 Axios Configuration and JWT Interceptors

File: src/services/axiosInstance.js — This is the most important service file. All other services import from here.

- Create axios instance with baseUrl from import.meta.env.VITE_API_URL.
- Add request interceptor: before every request, read the JWT token from authStore (Zustand) and add it to the Authorization header as 'Bearer {token}'.
- Add response interceptor: if any response returns HTTP 401, automatically call authService.refreshToken() with the stored refresh token, update the store with the new access token, and retry the original failed request. If refresh also fails, call authStore.logout() and redirect to /login.
- Add response interceptor for error formatting: extract error message from response.data.message or show generic 'Something went wrong'.

3.7 SignalR Chat Integration

File: src/services/chatService.js — SignalR connection management.

- Import HubConnectionBuilder from @microsoft/signalr.
- In connectSignalR() function: create connection to VITE_API_URL + '/hubs/chat' using withAutomaticReconnect(). Start connection. Register listener: connection.on('ReceiveMessage', callback). Register listener: connection.on('UserOnline', callback).
- In sendMessage(conversationId, message): call connection.invoke('SendMessage', conversationId, message).
- In useSignalR.js hook: call connectSignalR on component mount. Call disconnectSignalR on component unmount (cleanup). Expose isConnected state and sendMessage function to ChatPage.jsx.

FRONTEND WEEKLY SCHEDULE

Talha — 2 Hours Daily | April 20 — May 20, 2026

Section 4 — Frontend 30-Day Daily Schedule

Week 1: Apr 20–26 — Foundation, Routing & Auth Pages

Goal: Project scaffolded, all npm packages installed, auth pages working (login, register, forgot password), JWT stored and sent with requests.

WEEK 1	Haseeb — ASP.NET Core Backend	Talha — React Frontend
Mon Apr 20		Create Vite + React project. Install ALL npm packages from Section 3.2. Set up Tailwind CSS (init, configure tailwind.config.js, add directives to index.css). Create complete folder structure (all empty files). Push to Git.
Tue Apr 21		Configure axiosInstance.js with baseURL from .env. Set up Zustand authStore.js with user, token, isAuthenticated state. Create tokenUtils.js for localStorage token management. Create ProtectedRoute.jsx component. Set up App.jsx with React Router and all route paths (placeholder pages for now).
Wed Apr 22		Build Navbar.jsx — logo, nav links (Find Tutors, How It Works, For Tutors), Login/Register buttons when logged out, User avatar dropdown menu when logged in (Dashboard, Chat, Logout). Build Footer.jsx. Apply TutorHub color scheme from design file.
Thu Apr 23		Build LoginPage.jsx with LoginForm.jsx — email and password inputs using react-hook-form, Zod validation, submit calls authService.login(), on success store token in authStore + redirect to dashboard, show toast on error. Build ForgotPasswordPage.jsx with email input form.
Fri Apr 24		Build RegisterPage.jsx with RegisterForm.jsx — role selection toggle (Student / Parent / Tutor), all required fields, Zod validation matching backend rules (min 8 chars, uppercase, number, special char). On success redirect to 'Check your email' page. Build VerifyEmailPage.jsx that reads token from URL and calls authService.verifyEmail.

Sat Apr 25		Build ResetPasswordPage.jsx — new password + confirm password with Zod validation. Add request interceptors to axiosInstance.js — JWT header attachment. Add 401 response interceptor — refresh token flow. Test: login works, JWT is stored, protected routes redirect unauthenticated users.
Sun Apr 26		Sync with Haseeb. Test auth flow end-to-end against his local backend: Register → Verify Email → Login → Receive JWT → Access protected /dashboard/student page. Fix CORS issues if any. Create .env file with correct VITE_API_URL pointing to localhost:5000.

Week 2: Apr 27 — May 3 — Homepage, Search & Tutor Profile

Goal: Core discovery feature working. Users can search tutors, apply filters, and view full tutor profiles.

WEEK 2	Haseeb — ASP.NET Core Backend	Talha — React Frontend
Mon Apr 27		Build HomePage.jsx — hero section (big headline, SearchBar.jsx, CTA buttons), 'How It Works' section (3 steps: Search, Book, Learn), Featured Tutors section (use static data for now, replace with API later), stats row (2500+ tutors, 10000+ sessions). Apply design from HTML file.
Tue Apr 28		Build SearchBar.jsx with autocomplete — text input, on change (debounce 300ms) call searchService.getAutocomplete(), show dropdown list of suggestions. Build FilterPanel.jsx — price slider, rating checkboxes, subject multi-select (react-select), teaching mode radio buttons, city input. Build SortDropdown.jsx.
Wed Apr 29		Build SearchPage.jsx — layout (FilterPanel on left, results on right). Connect to searchService.searchTutors(filters) using React Query useQuery. Show LoadingSpinner while loading. Build TutorCard.jsx — avatar (Avatar.jsx), name, TutorBadge (if verified), star rating, subjects tags, price range, location, experience, View Profile and Message buttons.
Thu Apr 30		Add pagination to SearchPage.jsx using usePagination hook and Pagination.jsx component. Add 'X tutors found' count above results. Connect filter changes to re-fetch search results. Make FilterPanel collapse on mobile. Test search with various filter combinations against Haseeb's API.

Fri May 1		Build TutorProfilePage.jsx — top section (large avatar, name, verified badge, rating, location stats, Book Session and Message buttons). Build TutorAvailabilityGrid.jsx — 7-column weekly calendar grid showing green (available) and gray (booked) slots. Build TutorReviewList.jsx — star breakdown bar chart, individual review cards.
Sat May 2		Add 'About' section, qualifications table, experience, hourly rate, teaching mode to TutorProfilePage. Add demo video placeholder section. Connect TutorProfilePage to tutorService.getTutorProfile(id) using React Query. Show loading skeleton while profile loads.
Sun May 3		Sync with Haseeb. Test search and profile pages against real backend data. Ensure TutorCard data matches API response shape. Confirm autocomplete suggestions work. Fix any data mapping issues between backend DTOs and frontend display.

Week 3: May 4–10 — Booking Flow & Payments

Goal: Complete booking flow working end-to-end: select session details, proceed to Stripe payment, see confirmation.

WEEK 3	Haseeb — ASP.NET Core Backend	Talha — React Frontend
Mon May 4		Build BookSessionPage.jsx as a multi-step form using useBookingForm hook. Step 1: Subject dropdown (from tutor's subjects), Session Mode (Online/In-Person), Date picker (react-datepicker, disable past dates and dates too far ahead). Step 2: Time slot selection from tutor availability. Step 3: Duration buttons (30/60/90/120 min). Show PriceBreakdown.jsx (fee + 10% platform fee + total) updating live.
Tue May 5		Connect BookSessionPage to bookingService.createBooking(). On successful booking creation, redirect to PaymentPage.jsx with bookingId. Build CancelBookingModal.jsx reusable component. Build BookingStatusBadge.jsx with color coding.
Wed May 6		Build PaymentPage.jsx — left side: booking summary (tutor name, date, time, duration, subject). Right side: PriceBreakdown.jsx + StripeCardElement.jsx. Wrap page in Stripe Elements provider using loadStripe(VITE_STRIPE_PK). On submit: call paymentService.createPaymentIntent(bookingId), confirm payment with Stripe.js

Thu May 7		stripe.confirmCardPayment, call paymentService.confirmPayment.
Fri May 8		Build PaymentSuccessPage.jsx — success animation, booking details summary, buttons: View Booking, Go to Dashboard, Chat with Tutor. Build PaymentHistoryPage.jsx — table of all payments with amount, booking details, status, date. Connect to paymentService.getPaymentHistory().
Sat May 9		Build MyBookingsPage.jsx with 3 tabs (Upcoming, Past, Cancelled) using BookingCard.jsx. Upcoming tab: show Join Session button for online sessions (meeting link), Cancel button, Reschedule button. Past tab: show Leave Review button if no review submitted. Cancelled tab: show cancellation reason.
Sun May 10		Build ReviewForm.jsx — StarRating.jsx (clickable 1-5 stars), comment textarea (max 500 chars), submit button. Connect to reviewService.submitReview(). Show ReviewForm in a Modal.jsx after clicking 'Leave Review' on past booking. After submission, refresh booking data.
		Full booking flow integration test with Haseeb. Test: Search tutor → View Profile → Click Book Session → Select details → Pay with Stripe test card 4242 4242 4242 4242 → See PaymentSuccessPage → View booking in MyBookingsPage. Fix any issues in the flow.

Week 4: May 11–17 — Dashboards, Chat & Admin

Goal: All role-specific dashboards complete. Real-time chat working. Admin panel built.

WEEK 4	Haseeb — ASP.NET Core Backend	Talha — React Frontend
Mon May 11		Build DashboardLayout.jsx with Sidebar.jsx. Create role-aware sidebar: Student sees (Dashboard, My Bookings, Payments, Chat, Settings). Tutor sees (Dashboard, Profile, Availability, Documents, Bookings, Earnings, Chat, Settings). Admin sees (Dashboard, Users, Approvals, Bookings, Analytics, Subjects). Build StudentDashboard.jsx — upcoming bookings count, quick action buttons, recent chat previews.
Tue May 12		Build TutorDashboard.jsx — upcoming sessions list, total earnings this month, pending booking requests count, average rating display, profile completeness progress bar. Build TutorProfileEditPage.jsx — form with all editable fields: headline, bio, education, experience years, city,

		hourly rate min/max, teaching mode, subjects multi-select. Connect to <code>tutorService.updateTutorProfile</code> .
Wed May 13		Build <code>TutorAvailabilityPage.jsx</code> — 7-day grid, each day has hour slots. Click a slot to toggle available/unavailable. Save changes button calls <code>tutorService.setAvailability()</code> . Build <code>TutorDocumentsPage.jsx</code> — <code>FileUpload.jsx</code> component for government ID and education certificates. Show current document status badge (Pending / Approved / Rejected) for each uploaded document.
Thu May 14		Build <code>TutorBookingsPage.jsx</code> — pending booking requests with Confirm and Reject buttons. Confirmed upcoming sessions. Past sessions. Connect to <code>bookingService.confirmBooking</code> and <code>cancelBooking</code> . Build <code>TutorEarningsPage.jsx</code> — earnings chart (LineChart from recharts) with period filter (Daily/Weekly/Monthly). Payout history table. Request Payout button.
Fri May 15		Build <code>ChatPage.jsx</code> — two-panel layout. Left: <code>ConversationList.jsx</code> (all conversations, last message, unread badge). Right: <code>ChatWindow.jsx</code> (messages as bubbles, own messages right-aligned in blue, other messages left-aligned in gray). Bottom: <code>MessageInput.jsx</code> with send button. Connect <code>useSignalR</code> hook for real-time messages.
Sat May 16		Build <code>AdminDashboard.jsx</code> — row of <code>MetricCard.jsx</code> components (Total Users, Total Tutors, Bookings Today, Revenue Today). Add LineChart (recharts) for bookings over time. Add PieChart for top subjects. Build <code>AdminUsersPage.jsx</code> — data table with search, role filter. <code>UserTableRow.jsx</code> with suspend/activate buttons. Connect to <code>adminService</code> .
Sun May 17		Build <code>AdminTutorApprovalsPage.jsx</code> — list of pending applications. <code>TutorApplicationCard.jsx</code> with document link viewer, Approve button (green) and Reject button (red) with reason input. Build <code>AdminBookingsPage.jsx</code> — all bookings table with cancel action. Sync with Haseeb — test admin flows end to end.

Week 5: May 18–20 — Polish, Performance & Deployment Prep

Goal: App is polished, mobile responsive, error-handled, and ready for deployment.

WEEK 5	Haseeb — ASP.NET Core Backend	Talha — React Frontend
Mon May 18		Mobile responsiveness pass — test all pages on 375px viewport. Fix Navbar collapse to hamburger menu. Make FilterPanel slide-in drawer on mobile. Make BookingCard stack vertically on mobile. Add NotificationBell.jsx to Navbar with useNotifications hook. Build NotificationsPage.jsx. Build ProfileSettingsPage.jsx with edit personal info and change password forms.
Tue May 19		Error handling pass — add ErrorMessage.jsx to all pages that fetch data. Add EmptyState.jsx for empty search results, empty bookings, empty chat. Add ConfirmDialog.jsx before all destructive actions (cancel booking, delete review). Test with network offline to verify graceful error messages. Add page titles (document.title) for each page.
Wed May 20		Production build test — run npm run build. Fix any TypeScript or import errors. Check bundle size in build output. Run npm run preview to test production build locally. Confirm .env.production values are set correctly. Create ParentDashboard.jsx and ChildProfilesPage.jsx for parent role.

PART C — DEPLOYMENT

Both Haseeb & Talha | May 20, 2026

Section 5 — Full Deployment Guide

Deployment is done on the last day (May 20) after both backend and frontend are production-ready. Both team members work together during this session.

5.1 Pre-Deployment Checklist

Haseeb — Backend Checklist:

- All API endpoints tested in Postman and return correct responses.
- `dotnet publish --configuration Release` produces zero errors.
- All sensitive config (JWT secret, Stripe secret, SendGrid key, Azure Storage key) stored in Azure Key Vault — NOT in `appsettings.json`.
- Database migrations are up to date and DataSeeder has run.
- CORS is configured to only allow the production frontend domain.
- Swagger is disabled in Production environment (optional for security).
- Serilog is writing logs (can use Application Insights for Azure).

Talha — Frontend Checklist:

- `npm run build` completes with no errors.
- `.env.production` has correct `VITE_API_URL` (pointing to Azure backend URL) and `VITE_STRIPE_PK` (Stripe live publishable key).
- All API calls use `VITE_API_URL` environment variable (no hardcoded localhost URLs).
- All `console.log` statements removed.
- App tested in Chrome, Firefox, and mobile browser.

5.2 Backend Deployment — Azure App Service

Step	Action	Notes
1	Create Azure account at portal.azure.com (if not done yet)	Use student email for free credits
2	Create Resource Group named TutorHub-RG in region closest to users	All resources go in this group
3	Create Azure SQL Database — Basic tier (\$5/month or use free tier)	Note the server name and admin credentials
4	Update connection string in Azure App Service Configuration to point to Azure SQL Database	Never hardcode credentials
5	Create Azure Blob Storage account for document and image uploads	Get connection string from Access Keys
6	Create Azure Key Vault and add all secrets: <code>JwtSecretKey</code> , <code>SendGridApiKey</code> , <code>StripeSecretKey</code> , <code>BlobStorageConnection</code>	Backend reads from Key Vault at runtime

7	Create Azure App Service — Runtime stack: .NET 8, OS: Windows or Linux, Pricing: F1 Free tier for start	Note the default URL (yourapp.azurewebsites.net)
8	In Visual Studio: Right-click TutorHub.API project → Publish → Azure → Azure App Service → Select your App Service → Publish	Or use GitHub Actions CI/CD
9	In Azure App Service Configuration tab, add all environment variables that override appsettings.json (ConnectionStrings__DefaultConnection, Jwt__SecretKey, etc.)	These override appsettings.json values
10	Run EF Core migrations on Azure database: dotnet ef database update --connection 'AzureConnectionString'	Run from local terminal
11	Test backend at https://yourapp.azurewebsites.net/swagger to confirm API is live	All endpoints should load
12	Configure Stripe webhook URL in Stripe Dashboard to point to https://yourapp.azurewebsites.net/api/payment/webhook	Required for payment confirmation

5.3 Frontend Deployment — Azure Static Web Apps

Step	Action	Notes
1	Push React code to GitHub repository	Make sure .env is in .gitignore
2	In Azure Portal: Create Static Web App resource	Choose Free tier
3	Connect to your GitHub repository during Static Web App creation — Azure auto-creates GitHub Action CI/CD workflow	Choose main branch
4	Set build configuration: App location = '/', Output location = 'dist', API location = leave empty	Vite outputs to /dist folder
5	In Azure Static Web App Configuration: Add environment variables VITE_API_URL = https://yourbackend.azurewebsites.net and VITE_STRIPE_PK = your Stripe publishable key	These are injected at build time
6	Push any commit to GitHub — GitHub Action automatically builds and deploys React app	Watch the Actions tab in GitHub
7	Create staticwebapp.config.json in project root with fallback route for React Router: { 'navigationFallback': { 'rewrite': '/index.html' } }	Prevents 404 on direct URL access
8	Test frontend at https://yourapp.azurestaticapps.net — confirm all pages load and API calls reach backend	Test login, search, booking flow
9	Configure custom domain if available (optional) — Azure provides free SSL certificate	DNS settings at your domain registrar
10	Test SignalR chat in production — confirm WebSocket connection establishes to the Azure backend URL	Browser DevTools Network tab

5.4 Post-Deployment Testing Checklist

After both backend and frontend are deployed, run these tests together:

7. Register a new Student account → Receive verification email → Verify email → Login.
8. Register a new Tutor account → Complete profile → Upload documents → Wait for admin approval.
9. Login as Admin (use seeded admin credentials) → Approve the tutor application.
10. Login as Student → Search for tutors → Filter results → View a tutor profile.
11. Book a session with the tutor → Pay with Stripe test card → See confirmation.
12. Login as Tutor → Confirm the pending booking.
13. Open two browser windows (one as Student, one as Tutor) → Test real-time chat messaging.
14. Login as Admin → View analytics dashboard → See the test booking and revenue.
15. Test mobile responsiveness on an actual phone.
16. Test password reset flow from email.

5.5 Best Practices Followed Throughout This Project

Backend Best Practices (Haseeb)	Frontend Best Practices (Talha)
Clean Architecture — no circular dependencies between layers	Component-based architecture — each component does one thing
Repository Pattern — data access separated from business logic	Custom hooks — extract stateful logic from UI components
DTOs for all API inputs and outputs — never expose entities directly	Zod validation — validate all user inputs before API call
FluentValidation for all incoming request data	React Query — server state separated from client state
Async/await throughout — no blocking calls	Zustand for global state — only store auth and shared data
JWT with refresh token rotation — secure session management	Axios interceptors — centralized JWT management
BCrypt cost factor 12 for password hashing	Environment variables — no hardcoded API keys
CORS restricted to frontend domain only	ProtectedRoute — route-level authorization guards
Rate limiting — 100 requests/minute prevents abuse	Loading and error states — every async operation handled
Serilog structured logging — searchable production logs	Mobile-first CSS — Tailwind responsive classes
EF Core with parameterized queries — prevents SQL injection	Lazy loading — only fetch data when needed
Swagger documentation — all endpoints documented	Code splitting — React.lazy for large pages
Global exception middleware — consistent error responses	Toast notifications — user always knows what happened
Audit logs for all admin actions — compliance ready	Confirmation dialogs — prevent accidental destructive actions

Section 6 — Milestone Tracker

Milestone	Target Date	Success Criteria
Project Setup Complete	Apr 20, 2026	Both projects run locally, all packages installed, Git repos set up
Auth System Working	Apr 26, 2026	Register, login, JWT, email verification all functional
Tutor Module Complete	May 3, 2026	Profiles, documents, availability, and search working end-to-end
Booking & Payment Live	May 10, 2026	Full booking flow with Stripe payment in test mode
Chat & Admin Complete	May 17, 2026	SignalR chat working, admin panel functional, all dashboards built
Production Deployment	May 20, 2026	App live on Azure, all smoke tests pass, real data in database

Good luck, Haseeb and Talha!

2 hours a day. 30 days. Follow the plan. TutorHub will be live.